INTEGRATED PROJECT REPORT

Predictive Auto Scaling: Mini-to-Major Project on Workload Forecasting, Hysteresis and Safety-Aware Capacity Planning

Department of Computer Science and Engineering
National Institute of Technology, Warangal
Academic Year: 2026-27

Field	Details
Student Name / Roll No.	Dharmendra Kumar Gupta — 26CSM1R35 Satyam Kumar Azad — 26CSM1R22
Mentor	Dr. Manoj Kumar Patra
Project Type	Major Project — simulation-backed research prototype
Implementation Status	Local simulation and reproducible benchmark; live cloud deployment is future work
Academic Year	2026-27

Abstract

Cloud applications experience demand changes that can cause excess provisioning, slow response, and repeated scaling actions. This project upgrades an initial mini-project into a reproducible benchmark study of a reactive threshold autoscaler and a predictive controller. The predictive controller forecasts short-term request demand using a trend-adjusted moving average, adds a safety margin, estimates required capacity, and combines the decision with hysteresis, cooldowns, and instance warm-up delay. The extended experimental suite includes six synthetic workload profiles, two controller strategies, request/instance/latency traces, scenario-level metrics, and automated tests. The simulator measures provisioning, scale actions, average and tail latency, SLA violations, forecast error, and estimated instance-minute cost. Results are explicitly labeled as simulated and are not represented as real cloud measurements. The project package includes runnable source code, generated datasets, CSV outputs, plots, report, test suite, and deployment roadmap.

1. Introduction and Problem Statement

1.1 Background

Auto scaling changes the number of compute instances in response to application demand. Reactive systems react after load crosses a threshold, while predictive systems attempt to provision capacity before the full demand arrives. A useful design must balance latency, capacity cost, stability, and the delay required to bring new instances into service.

1.2 Problem statement

A single CPU threshold can react late to abrupt spikes and can oscillate when demand fluctuates around a boundary. Scaling decisions that ignore forecast uncertainty, warm-up time, and service-level objectives can produce poor trade-offs. The project evaluates whether short-horizon demand prediction plus conservative decision controls improves the resource/latency trade-off under varied workloads.

1.3 Objectives

- Implement a deterministic, reproducible simulation of reactive and predictive scaling.
- Generate six workload profiles covering stable, ramp, spike, burst, noisy, and recovery behaviour.
- Model capacity limits, instance warm-up delay, cooldown, hysteresis, and a simple latency/SLA proxy.
- Benchmark cost proxy, provisioning, scale actions, mean/p95/p99 latency, SLA violations, and forecast error.
- Produce graph-rich outputs and automated tests so experiments can be repeated.
- Define a staged route toward real cloud validation without claiming that it has already been deployed.

2. Existing Mini Project Review

The earlier mini-project used one 360-minute synthetic trace and reported a reactive baseline against a predictive controller. The previously reported values are preserved below for historical comparison. The major-project experiment is broader and uses a different workload suite and simulator implementation, so its results should not be treated as a strict apples-to-apples rerun of those earlier numbers.

Earlier mini-project metric	Reactive baseline	Predictive + hysteresis
Average instances	1.933	1.794
Peak instances	4	4
Scaling actions	21	17
SLA violations	0	0
Estimated cost units	6.96	6.46

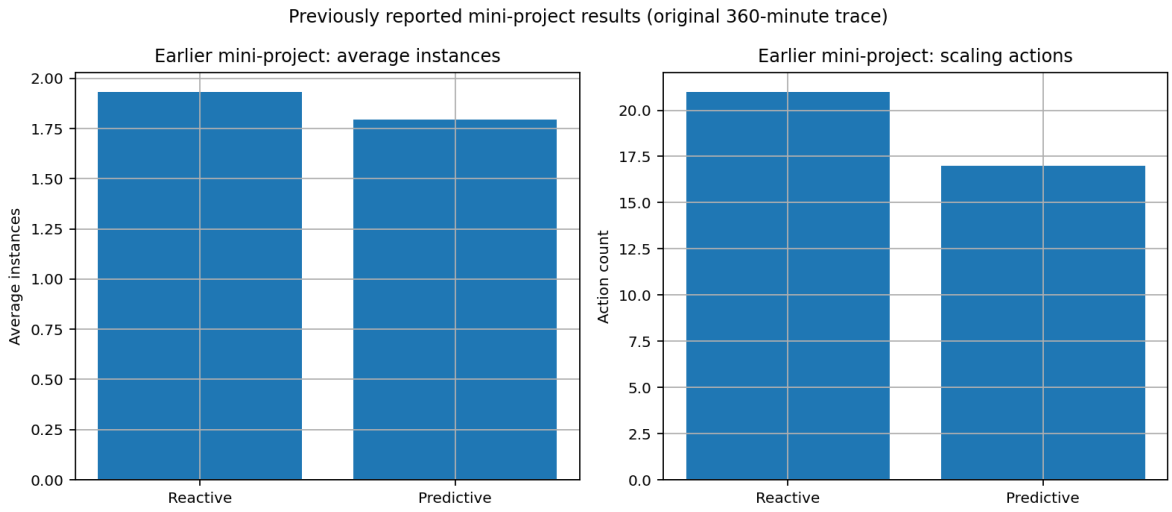


Figure 1. Previously reported mini-project outcomes; retained as historical baseline.

3. Literature and Design Rationale

The design follows common autoscaling themes: threshold-based reactive control, workload forecasting, hysteresis to reduce oscillation, cooldowns to avoid repeated changes, and warm-up-aware capacity planning. This project is an engineering evaluation rather than a systematic literature review; final academic submission should replace or supplement this section with verified papers selected by the team and cited in the required institutional style.

- Reactive threshold control: simple and explainable, but reacts after a threshold crossing.
- Predictive control: estimates near-future demand, potentially starting provisioning earlier; quality depends on workload predictability.
- Hysteresis/cooldown: avoids rapid scale-in/scale-out reversals but can delay response if too conservative.
- Warm-up modelling: makes newly requested capacity unavailable until its startup delay expires.
- Multi-scenario evaluation: avoids relying on one trace and exposes controller strengths and weaknesses under different patterns.

4. Proposed Architecture and Methodology

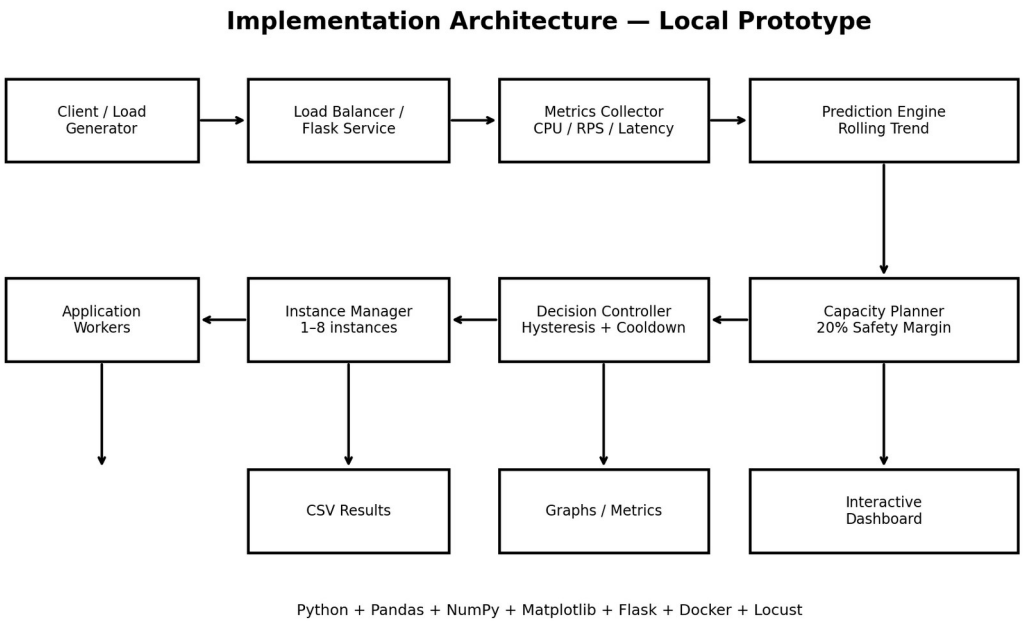


Figure 2. System implementation architecture carried forward from the mini-project.

The simulator accepts a time-indexed request-rate trace. At each interval it updates pending instances, estimates current utilization, optionally forecasts future demand, chooses a scaling action, applies warm-up and cooldown rules, and records the resulting capacity and response-time proxy. The same workload seed is used for both controllers to support a controlled comparison.

4.1 Controller logic

- Reactive baseline: scale out when measured utilization exceeds 70%; scale in after sustained low utilization below 30%; obey cooldown and instance bounds.
- Predictive controller: forecast near-term request rate from recent observations; calculate ceil(forecast × (1 + safety margin) / per-instance capacity); clamp desired capacity to minimum and maximum; require sustained overload before scaling out and sustained underload before scaling in.
- Warm-up: newly requested instances are pending for two simulation intervals before joining active capacity.
- Capacity limits: minimum one and maximum twelve instances in the major benchmark.
- Latency proxy: response time is computed from a simple queue-over-capacity model plus small deterministic noise; SLA threshold is 200 ms.

4.2 Key equations

Required instances = ceil(predicted request rate × (1 + safety margin) / capacity per instance). In the benchmark, capacity per instance is 220 requests/minute and the safety margin is 15%. The response-time equation is a simulation proxy, not a calibrated performance model: latency = 55 ms + 240 × max(0, demand / active capacity − 1) + noise.

4.3 Architecture / decision flow

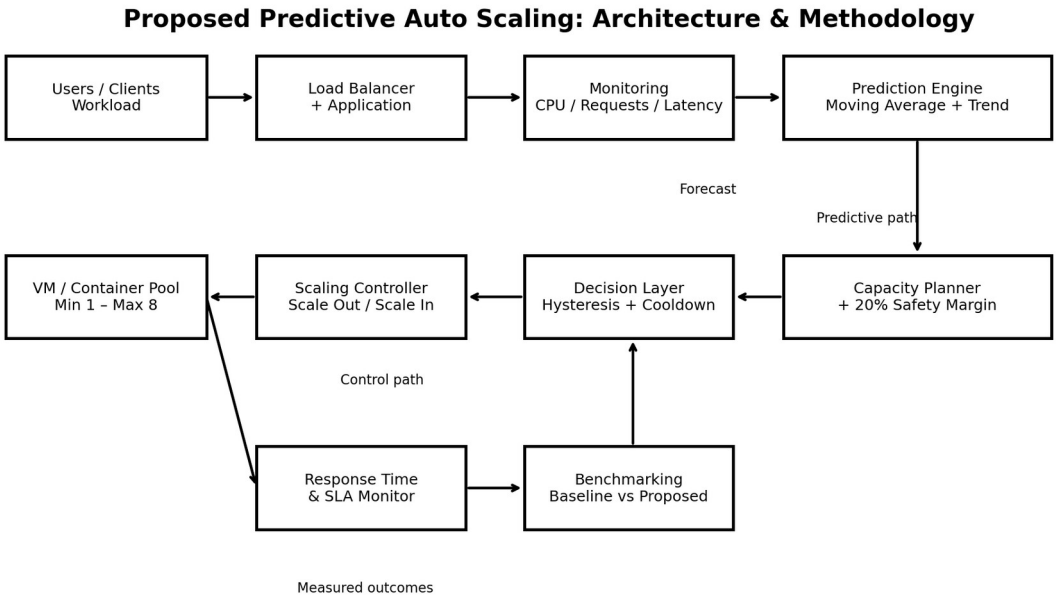


Figure 3. Proposed controller and decision methodology.

5. Experimental Setup and Benchmarking

Environment: Python 3, NumPy, Pandas, Matplotlib; CSV traces and fixed random seeds; no cloud billing or real service telemetry is involved. Each of six workload profiles runs through both controllers for 720 one-minute intervals. Per-instance capacity is 220 requests/minute, capacity is bounded to 1–12 instances, warm-up is two minutes, predictive safety margin is 15%, and the SLA proxy threshold is 200 ms.

Scenario	Purpose
Stable	Check whether controller avoids unnecessary capacity changes.
Gradual ramp	Assess proactive capacity as demand rises.
Sudden spikes	Expose reaction delay and warm-up effects.
Repeated bursts	Test repeated short bursts and cooldown behaviour.
Noisy fluctuation	Test sensitivity to irregular demand.
Spike and recovery	Evaluate scaling out and later scaling in.

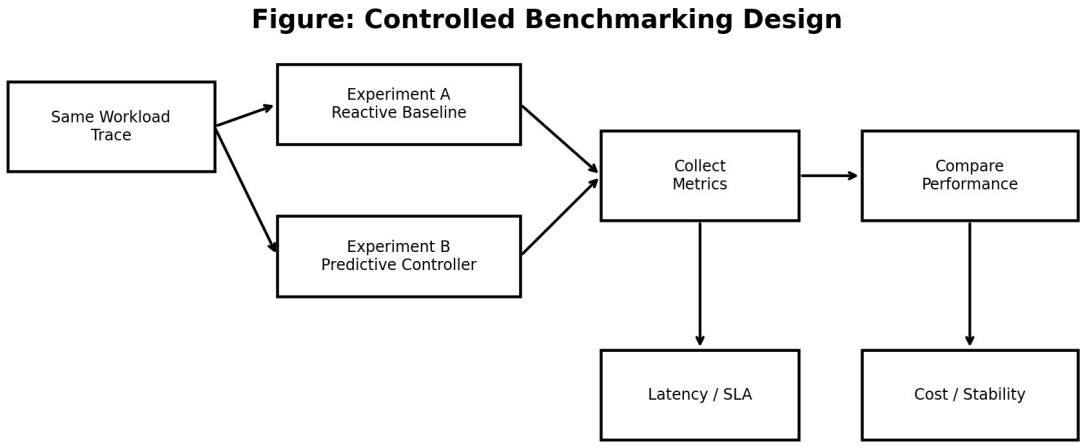


Figure 4. Experimental design and comparison pipeline.

5.1 Metrics

- Average active/provisioned instances and peak provisioned instances.
- Number of scale-out and scale-in actions.
- Average, p95, and p99 response-time proxy.
- SLA violation count and percentage of intervals above 200 ms.
- Mean absolute forecast error for predictive controller.
- Estimated instance-minutes as a transparent cost proxy; this is not a currency estimate.

6. Updated Results and Analysis

The following results are generated by the new major-project simulator. They are simulated benchmark outcomes and should be rerun on the target machine before submission. The CSV files in results/ contain scenario-level raw records and aggregated metrics.

Controller	Avg prov.	Peak	Actions	Avg ms	P95 ms	SLA %	Instance-min	MAE req/min
reactive	2.67	8	165	56.09	60.18	0.347	11534.0	nan
predictive	2.193	8	147	59.42	60.99	0.949	9474.0	36.56

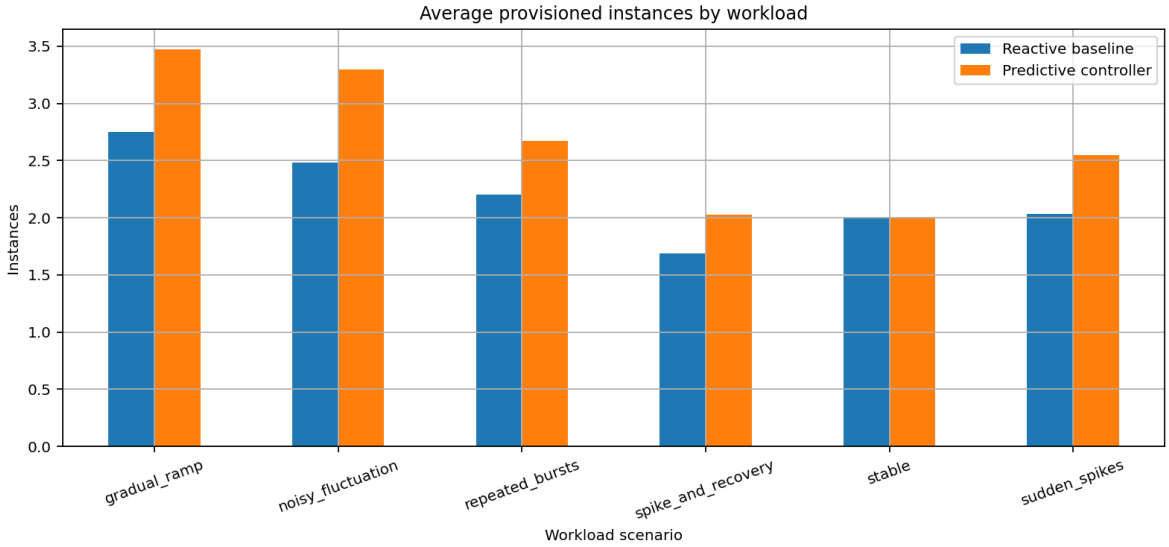


Figure 5. Average provisioned capacity by scenario.

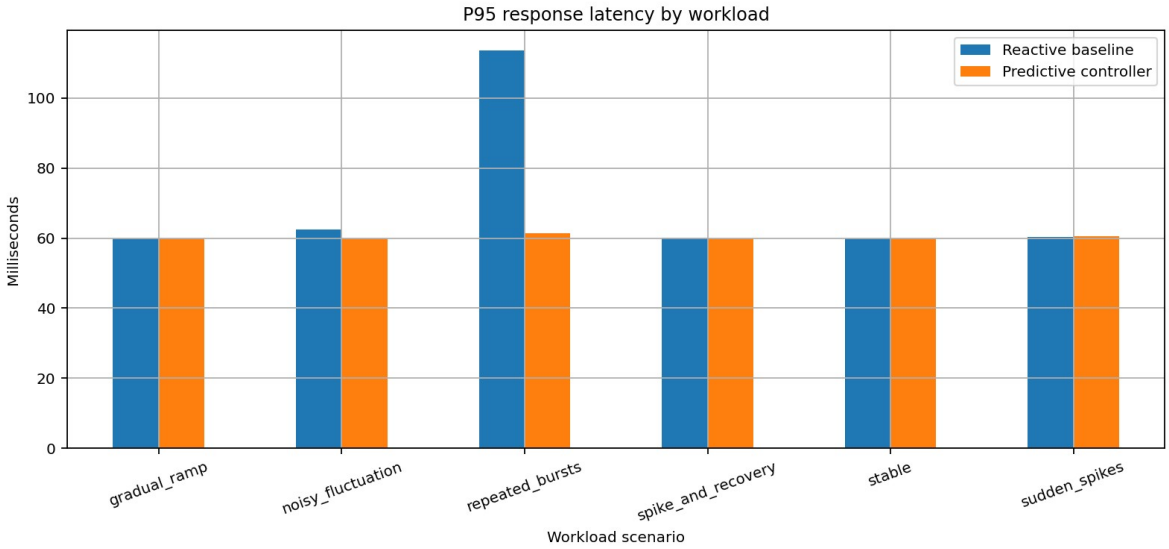


Figure 6. Tail latency comparison by scenario.

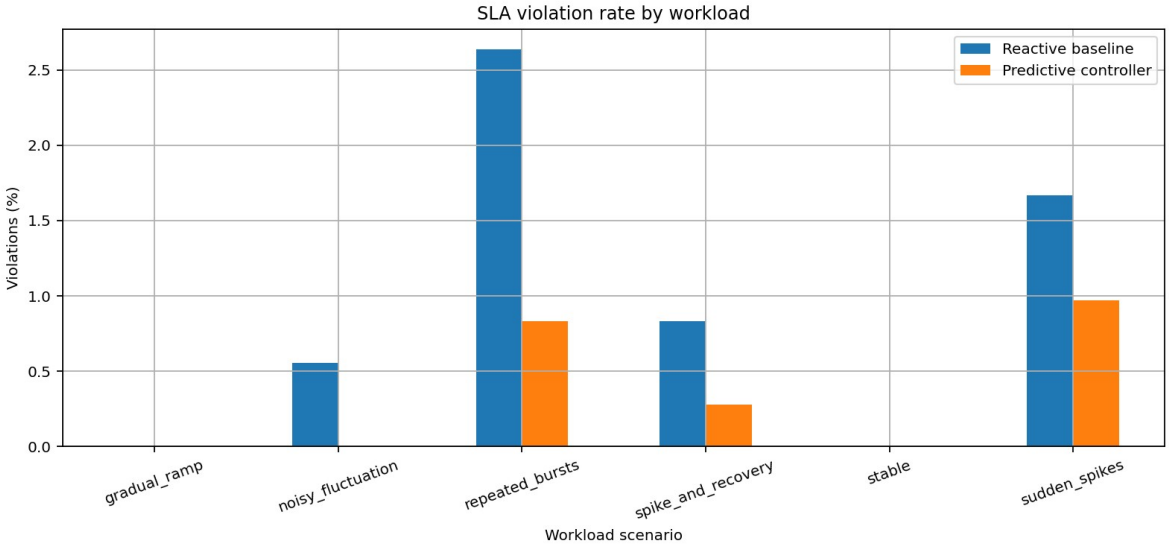


Figure 7. SLA violation rate by scenario.

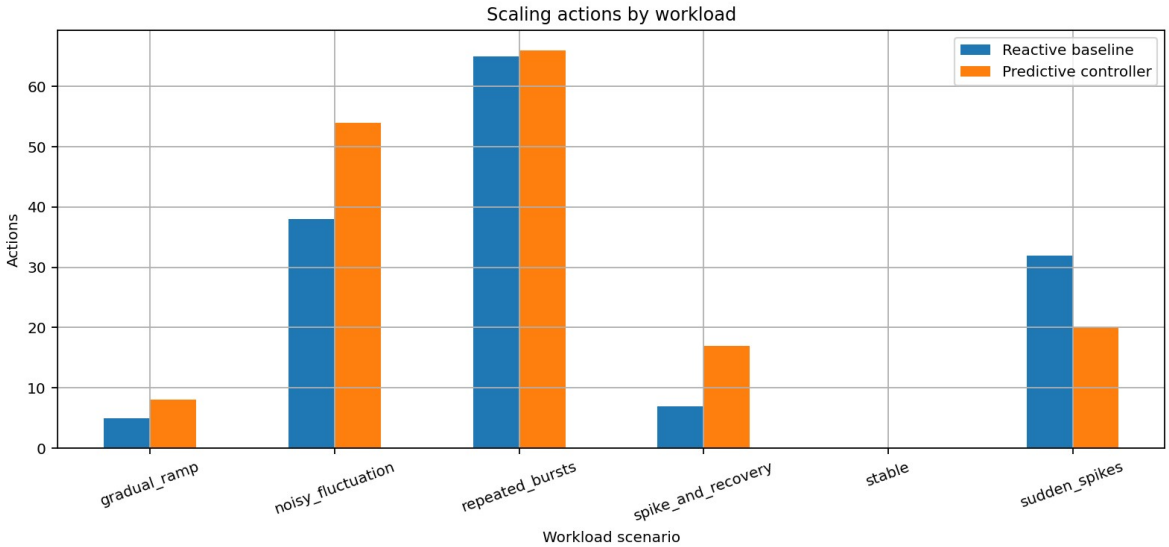


Figure 8. Scaling action count by scenario.

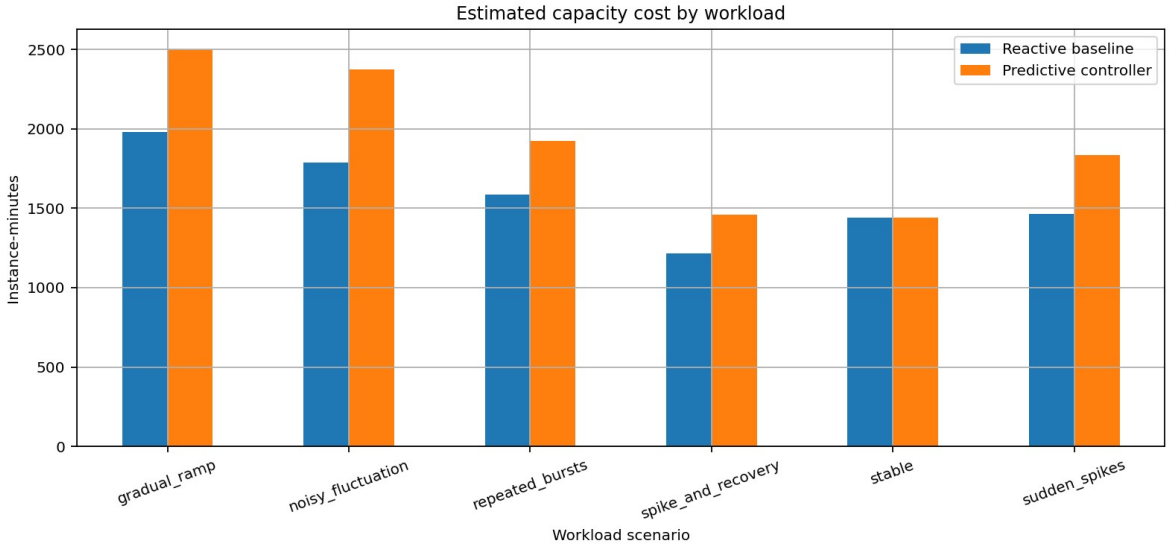


Figure 9. Estimated capacity-cost proxy by scenario.

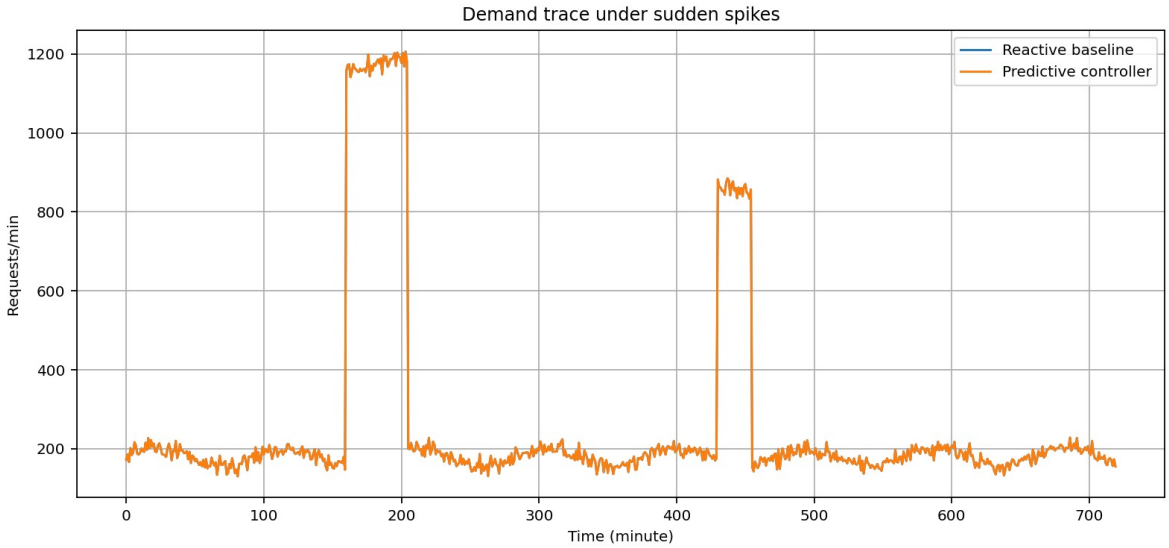


Figure 10. Sudden-spike demand trace.

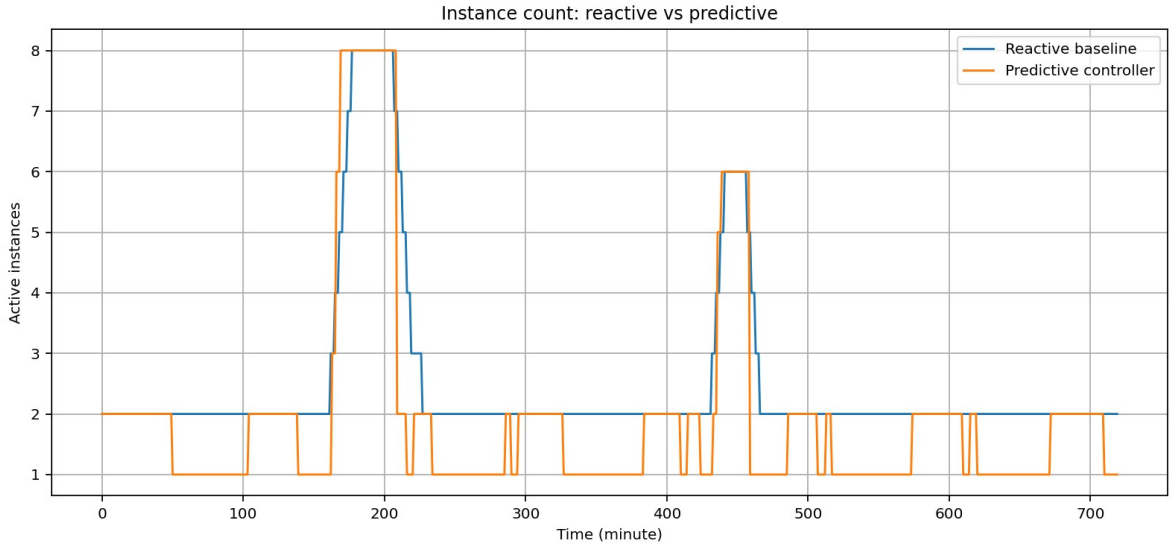


Figure 11. Active instance timeline during sudden spikes.

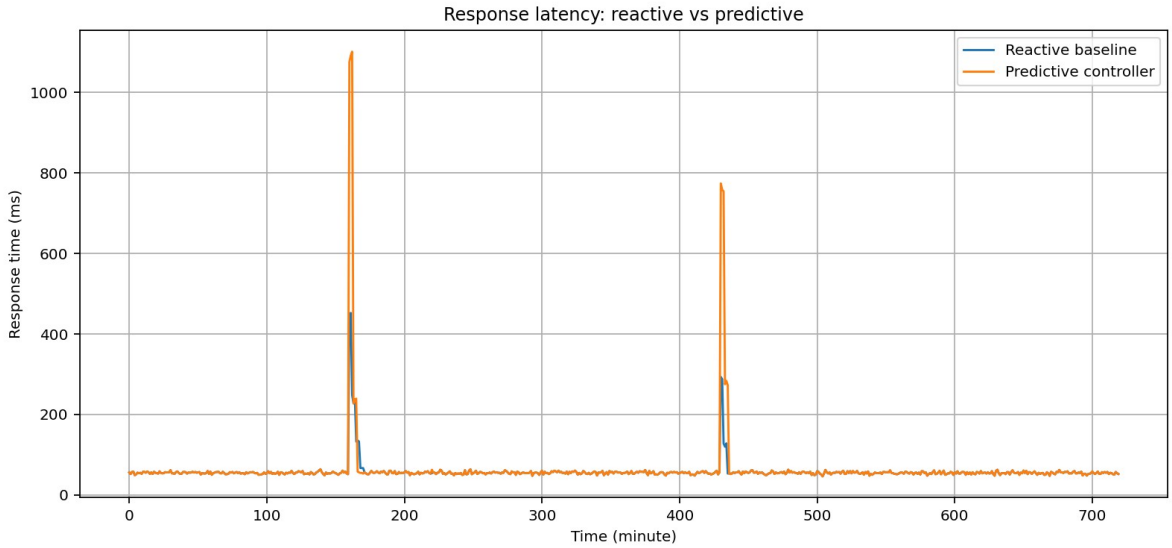


Figure 12. Response-time timeline during sudden spikes.

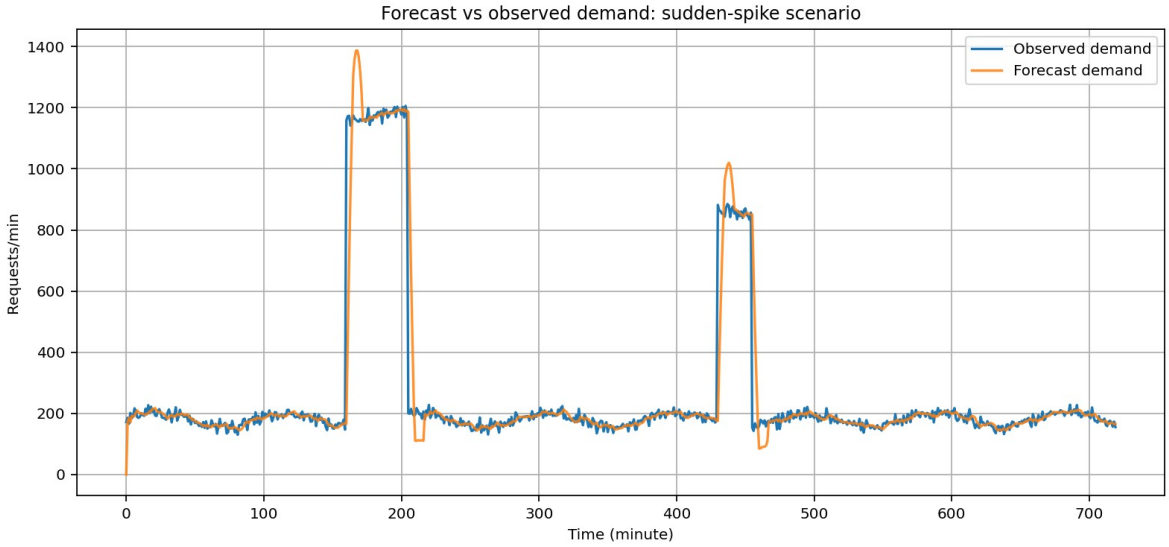


Figure 13. Predictive forecast against observed demand.

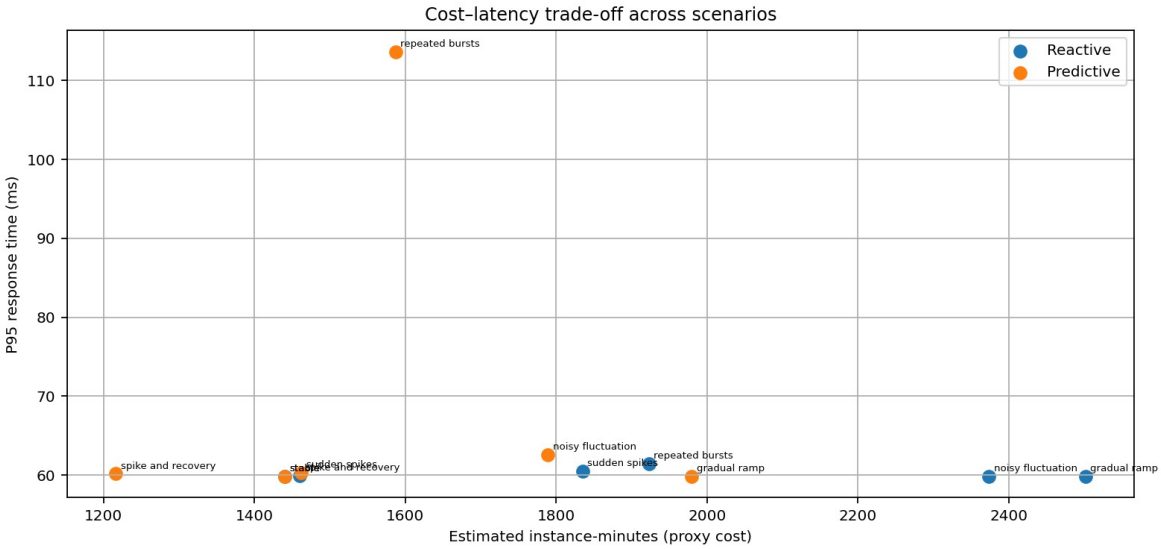


Figure 14. Cost–latency trade-off across workload scenarios.

6.1 How to interpret the results

Do not select a controller from one metric alone. Fewer instances or lower instance-minutes may indicate lower cost, but must be considered alongside tail latency and SLA violations. Fewer scaling actions may indicate stability, but can also indicate a controller that reacts too slowly. The scenario-level tables should be inspected to identify where the predictive strategy helps and where forecasting error or warm-up delay remains costly.

6.2 Previous versus major-project results

The earlier project's two-controller summary is retained in results/previous_vs_updated_summary.csv and Figure 1. The new suite expands from one trace to six generated workload classes and adds warm-up delay, p95/p99 latency, forecast error, per-scenario breakdowns, and test coverage. Because the workload and model have changed, do not directly subtract the old summary from the new overall metrics or present the difference as a controlled improvement. A future fair comparison should run both old and new controllers against identical traces, configuration, and simulator semantics.

7. Cases Addressed

- Stable workload: avoid unnecessary churn.
- Gradual increase: estimate near-term capacity before load continues to rise.
- Sudden spike: measure impact of reaction delay and warm-up.
- Repeated bursts: examine cooldown and oscillation behaviour.
- Noisy workload: examine sensitivity to short-term variation.
- Recovery/low demand: evaluate scale-in and capacity reduction.
- Capacity limit: cap at twelve instances and observe residual overload.
- Forecast error: compare predicted and observed request rates.

8. Scope and Limitations

- Workloads are synthetic and deterministic for reproducibility; they are not traces from a production service.
- Response time is a simplified queue-over-capacity proxy, not a full web server/load-balancer benchmark.
- Instance-minute cost is a relative proxy, not actual cloud billing.
- Only request rate is used for forecasting; CPU, memory, queue length, and error rates are not fused into the controller.
- Forecast uncertainty is not explicitly modelled as a confidence interval.
- Warm-up is fixed at two minutes and does not represent real VM startup distributions.
- The simulator is single-service and does not model database bottlenecks, network contention, containers, or multi-tier dependencies.
- The included Flask/Locust and Docker files are starting points; a cloud deployment and load-test validation are not claimed as completed.

9. Testing and Reproducibility

- Run: python src/run_experiments.py from the project root.
- Install dependencies: pip install -r requirements.txt.
- Run tests: python -m pytest tests/ (pytest may need to be installed separately).
- Outputs are written to results/; graphs are saved in diagrams/.
- Use fixed seeds to reproduce the same generated workloads.
- Keep raw CSVs and configuration with every report revision.

10. Implementation Roadmap to a Full Major Project

Phase	Deliverable	Acceptance evidence
1. Reproducible simulation	Current package: six workload profiles, two controllers, CSV results and graphs	Successful script run and tests
2. Stronger forecasting	Compare moving average with Linear Regression, Random Forest/XGBoost, and seasonal naive baseline	MAE/RMSE by scenario; train/test split with no leakage
3. Multi-metric control	Include CPU, memory, queue depth and request latency	Ablation study showing contribution of each metric
4. Statistical evaluation	Run 20+ seeds or bootstrap confidence intervals	Mean, standard deviation, confidence intervals
5. Real application load test	Flask service + Locust; collect actual throughput and latency	Raw load-test results and repeatable scripts
6. Cloud prototype	Deploy to a selected cloud autoscaling group or Kubernetes HPA/KEDA	Architecture, configuration, measured startup and cost
7. Safety and observability	Add action logs, rollback guardrails, min/max policy and dashboard	Audit log, failure tests, operational runbook
8. Final validation	Same workloads/configuration for all baselines; independent test traces	Reproducible tables, limitations, supervisor review

11. Future Scope

- Compare statistical and ML predictors (seasonal naive, linear regression, Random Forest, XGBoost, LSTM/GRU).
- Use prediction intervals or confidence-aware safety margins.
- Add multi-metric signals such as CPU, memory, queue depth, request errors, and latency.
- Calibrate the simulator with real load tests and empirical startup distributions.
- Evaluate on real workload traces and run repeated trials with confidence intervals.
- Deploy to a cloud managed instance group or Kubernetes HPA/KEDA; compare with provider-native autoscaling.
- Add a dashboard, audit logs, anomaly detection, policy guardrails, and cost/carbon-aware objectives.

12. Conclusion

This major-project version turns the earlier demonstration into a reproducible comparative study with multiple workload classes, warm-up-aware simulation, richer metrics, raw result files, and a graph set suitable for review. It provides a stronger research and engineering foundation, but it is not yet a production cloud deployment. The next essential steps are stronger baseline comparisons, repeated-seed statistical evaluation, real application load testing, and eventual cloud validation.

Appendix A: Project file map

- src/major_simulator.py — workload generation, controllers, simulation, metrics.
- src/run_experiments.py — benchmark entry point.
- tests/test_major_simulator.py — basic automated checks.
- results/scenario_metrics.csv and overall_metrics.csv — aggregate results.
- results/*_reactive.csv and results/*_predictive.csv — raw time series.
- diagrams/*.png — graphs and architecture diagrams.
- interactive/interactive_autoscaling_demo.html — original interactive demonstrator.

References and academic note

The current package preserves the project's engineering rationale but does not fabricate bibliographic entries. For a final academic submission, add verified research papers on reactive/predictive autoscaling, workload forecasting, hysteresis, cloud benchmarking, and the selected cloud platform. Record exact author/title/venue/year/DOI and cite them in the body using the institution's required style.